

lab3 Parser-26sp

lab3 ddl: 2026/4/29 23:59

1. Introduction

1.1 Why a compiler needs a parser?

The parser operates after the lexical analysis phase of the compiler, which breaks down the source code into individual words or tokens. The parser takes these tokens and checks them against the grammar of the language. This grammar defines how tokens can be combined to form valid statements and expressions. If the tokens follow the grammar rules, the parser will construct a parse tree, a hierarchical data structure that represents the syntactic structure of the source code.

1.2 What is **Bisonc++**?

Bisonc++ is a general-purpose parser generator converting grammar descriptions for LALR(1) context-free grammars into C++ classes whose members can parse such grammars.

1.3 Why use **Bisonc++** not **YACC**? And what's the difference between these two tools?

If you've read the textbook, you may find that the programming exercises in the textbook uses Yacc(yet another compiler compiler) to generate a syntax parser. In short, Yacc and Bisonc++ are both tools for generating syntax parsers. Bisonc++ is an extended version of Yacc, offering more features and better error handling. Bisonc++ is part of the GNU project and is free software, while Yacc is proprietary. Bisonc++ is more widely used in open-source projects due to its features and licensing.

2. Lab Description

In lab3, your task is to use **Bisonc++** to implement a parser for the Tiger language. In other words, you should write the correct rules in `tiger.y` file that describes a parser, which will generate a proper abstract syntax tree (AKA AST, you'll learn more about abstract syntax trees in Chapter 4) for every given tiger source file. Please refer to the *Tiger Language Reference Manual* (Appendix A) for the details of the syntax of Tiger Language.

The following files are related to this lab:

- `src/tiger/absyn/absync.*` The declarations and definitions of all abstract syntax classes
- `src/tiger/parse/tiger.y` The grammar file you must fill in.
- `src/tiger/parse/parser.*` The Parser class declaration.

- `src/tiger/errmsg/errmsg.*` The `ErrMsg` class, which is useful for producing error messages with file names and line numbers.
- `src/tiger/symbol/symbol.*` The `Symbol` class and `Symbol` table.

The testcases for this lab are in `testdata/lab3/testcases/` and the correct outputs of the testcases are in `testdata/lab3/refs/`

Remember that you will only need to fill in the file `tiger.y`. Moreover, your grammar should have as few shift-reduce conflicts as possible and no reduce-reduce conflicts at all.

Hints

- Some tokens such as `ID`, `STRING`, and `INT` will have semantic values. We have already set their semantic values for you in the function `Parser::lex()` in `src/tiger/parse/parser.h`.
- Normally, we will use `%token` directives to define tokens. However, when you want to define some precedence rules in directives like `%left`, you need to remove the corresponding token definition in `%token` directive because the precedence directives will also define tokens, or there will be a duplicate token definition error.
- Different from the textbook, we added a new expression class called `VoidExp` to refer to the empty expression, which is just wrapped in parentheses.
- We will use `std::list` instead of traditional C-list from now on. You are supposed to learn some basic usage of `std::list` and use it to generate correct AST in this lab.
- Recall what the teacher mentioned in class about how symbols are sorted according to their priorities.
- When you see these warnings on the right, instead of ignoring them, fix them.



```
stu@8e315dd3812d:~/tiger-compiler$ make build
mkdir -p build && cd build && cmake -DCMAKE_BUILD_TYPE=Release .. && make
-- Configuring done
-- Generating done
-- Build files have been written to: /home/stu/tiger-compiler/build
make[1]: Entering directory '/home/stu/tiger-compiler/build'
make[2]: Entering directory '/home/stu/tiger-compiler/build'
make[3]: Entering directory '/home/stu/tiger-compiler/build'
make[3]: Leaving directory '/home/stu/tiger-compiler/build'
make[3]: Entering directory '/home/stu/tiger-compiler/build'
[ 4%] Generating ../src/tiger/parse/parser.cc, ../src/tiger/parse/parserbase.h
[Warning] 15 Shift/Reduce conflict(s)
rule 75 (tiger.y, line 179): shifts at LBRACK, LBRACK,
rule 54 (tiger.y, line 145): shifts at TYPE, TYPE,
rule 56 (tiger.y, line 149): shifts at FUNCTION, FUNCTION,
rule 45 (tiger.y, line 130): shifts at RPAREN, RPAREN,
rule 45 (tiger.y, line 130): shifts at RPAREN,
rule 13 (tiger.y, line 87): shifts at ELSE,
rule 15 (tiger.y, line 89): shifts at ELSE,
rule 45 (tiger.y, line 130): shifts at RPAREN,
rule 45 (tiger.y, line 130): shifts at RPAREN,
rule 45 (tiger.y, line 130): shifts at RPAREN,
rule 45 (tiger.y, line 130): shifts at RPAREN,
[Warning] Terminal symbol(s) not used in productions:
[Warning] 308: actuals
[Warning] 309: nonemptyactuals
[Warning] 310: sequencing
[Warning] 313: one
[Warning] 314: oneormore
[Warning] 316: decs_nonempty
[Warning] 320: rec_nonempty
[Warning] 325: tyfields_nonempty
[Warning] 326: tyfield
```

Notice: Before you start this lab, you should carefully read chapter 3 of the textbook, and you may need to get some references from the [Bison++ manual](#) and the *Tiger Language Reference Manual* (Appendix A). As usual, please don't hesitate to reach out to the TA if you have any questions about this lab!

3. Environment

You will use the same code framework that you had set up when you worked on lab2. What you need to do now is to pull the latest update of the code framework if there are any. You may have to do some code-merging jobs.

! There have been some changes made to the lab3 branch, so please update the branch lab3 before coding.

Code block

```
1  shell% git add .
2  shell% git commit -m 'write_down_commit_msg'
3  shell% git pull origin lab3
4  shell% git checkout -b lab3
5  shell% git merge lab2
```

4. Grade Test

The lab environment contains a grading script named **scripts/grade.sh**, you can use it to evaluate your code, and that's how we grade your code, too. If you pass all the tests, the script will print a successful hint, otherwise, it will output some error messages. You can execute the script with the following commands.

Code block

```
1  shell% make gradelab3 or bash ./scripts/grade.sh lab3
2  shell% ...
3  shell% [^_^]: Pass** _#If you pass all the tests, you will see these messages.
4  shell% SCORE: 100
```

5. Submission Instructions

1. Using `make ziplab3` command to generate the zip file.

2. Hand in the zip file to gradescope website. (you can see previous documents for more information)

! Remember to submit before the deadline to avoid late penalties!

6. Key Takeaways

After completing Lab 3, you will have gained experience in the following parts:

- **Parsing:** Understanding and defining the grammar rules for the Tiger language using Bisonc++ with hands-on experience. Working with Bisonc++ to generate C++ classes based on LALR(1) context-free grammars helps you understand the concepts behind LALR(1) parsing and how to leverage Bisonc++ to build parsers for the specific language.
- **Abstract Syntax Tree (AST) Generation:** You will have experience in constructing an Abstract Syntax Tree (AST) for Tiger programs. The parser you implement will not only recognize valid Tiger code but also build a tree-like data structure that represents the program's syntactic structure. This AST serves as a foundation for further analysis, optimization, and code generation.

😊 So, dive in, explore, and enjoy the learning process!